Lesson End Project
Creating a React Application with Redux Store

Project agenda: To develop a React application using Vite that demonstrates the use of Redux Toolkit for managing application state, Context API for theme toggling, and asynchronous data fetching using createAsyncThunk. The project features a sports dashboard that retrieves and displays live match data from an external API, handles user login state, and allows theme switching between light and dark modes.

Description: You are tasked with building a React application using Vite that integrates Redux Toolkit and Context API to demonstrate scalable state management and real-time data handling. The application will use Redux slices to manage match data and user state, createAsyncThunk for asynchronous API calls, and Context API to control theme toggling across components. This project showcases modular architecture, async data flows, and clean separation of UI and business logic in a dynamic sports dashboard.

Tools required: Visual Studio Code and Node.js

Prerequisites: None

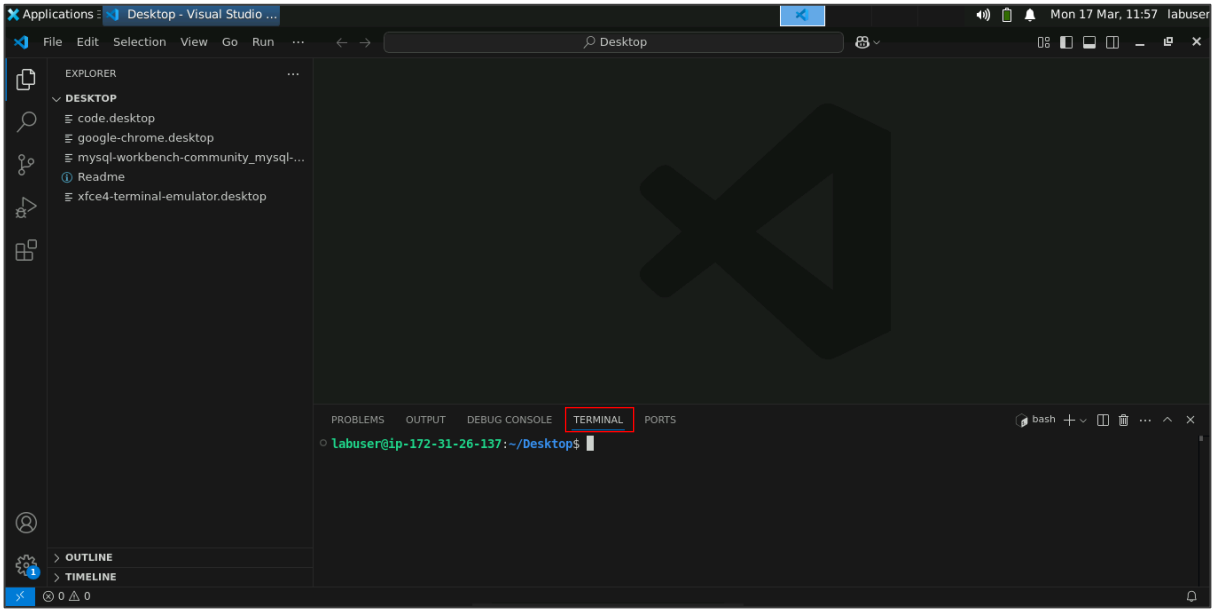
Expected deliverables: A fully functional React application built with Vite, demonstrating the use of Redux Toolkit for state management, Context API for theme control, and asynchronous data fetching with createAsyncThunk. The application will include modular Redux slices, live match data rendering, user session handling, and global theme toggling. It will showcase clean project structure, maintainable code, and an interactive UI, serving as a practical reference for combining Redux and Context in modern React applications.

Steps to be followed:

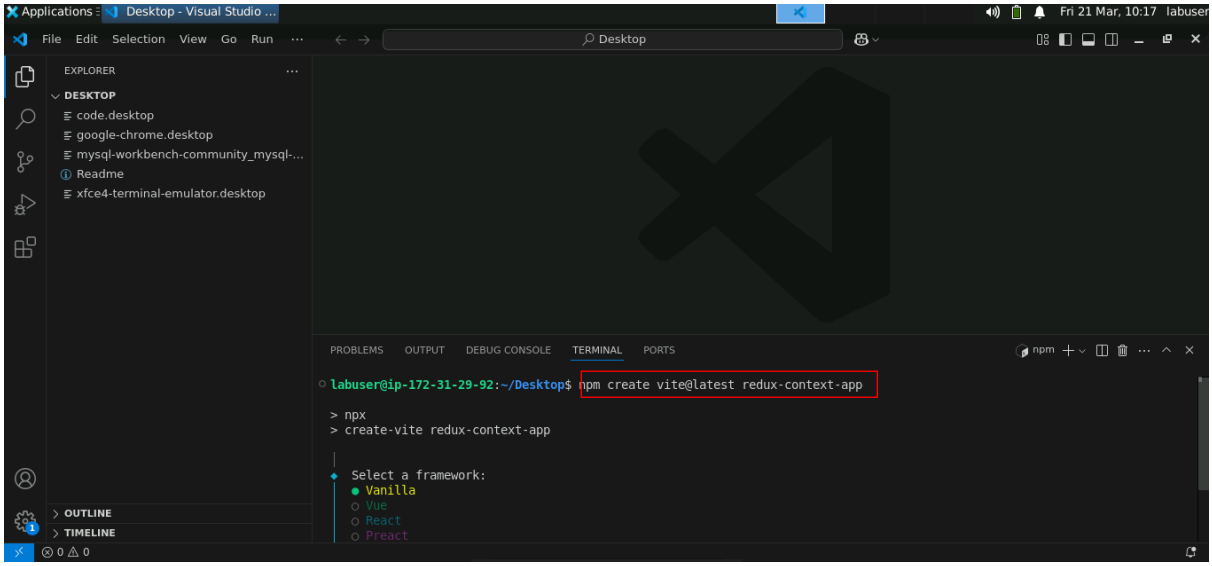
1. Set up a new React project using Vite
2. Set up Redux store with slices and async thunks
3. Implement Context API for theme toggling
4. Build App component with Redux and Context integration
5. Update main.jsx
6. Run and verify the application

Step 1: Set up a new React project using Vite

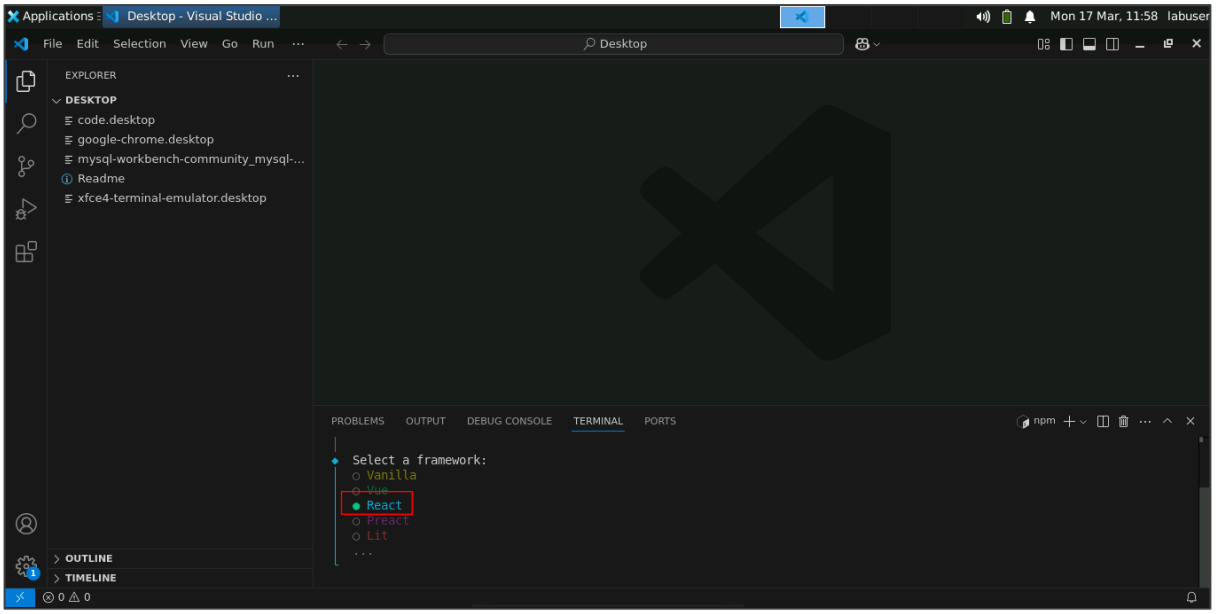
1.1 Navigate to VS Code and open the terminal



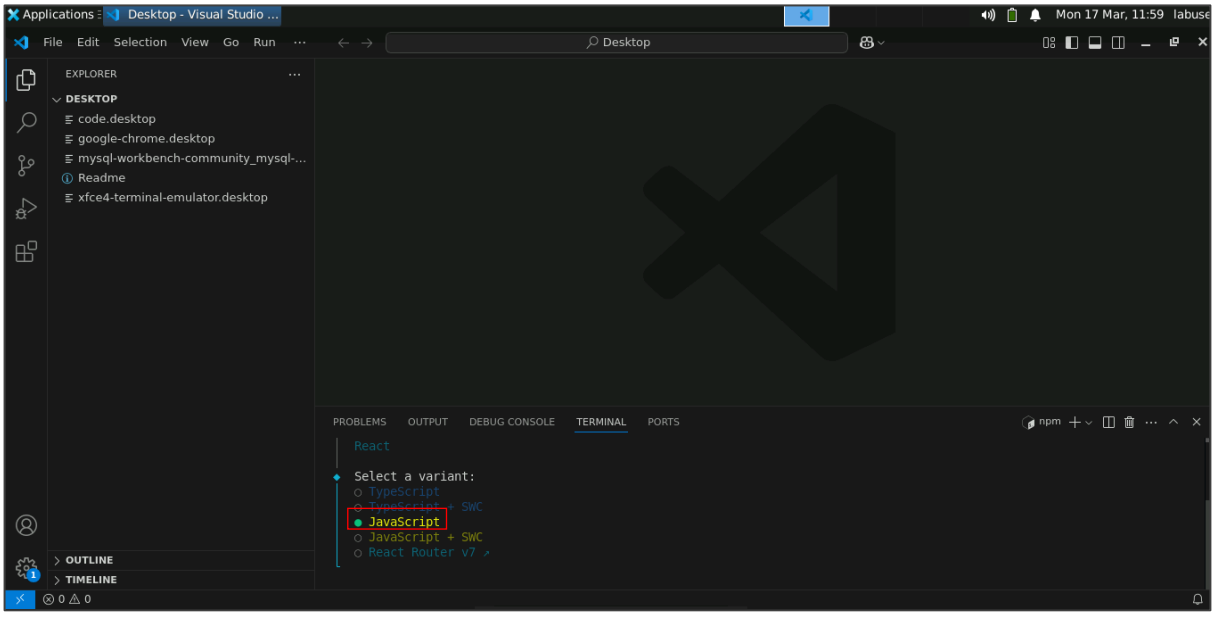
1.2 Run the following command to create a new React project using Vite:
npm create vite@latest redux-context-app



1.3 Select **React** and press enter

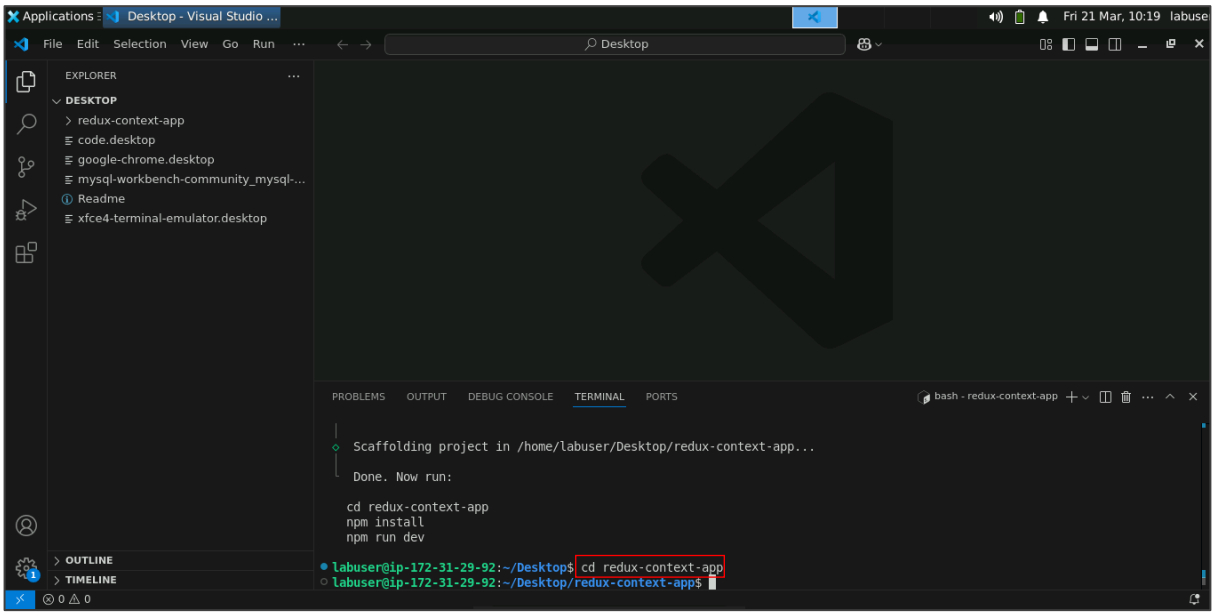


1.4 Select **JavaScript** and press enter



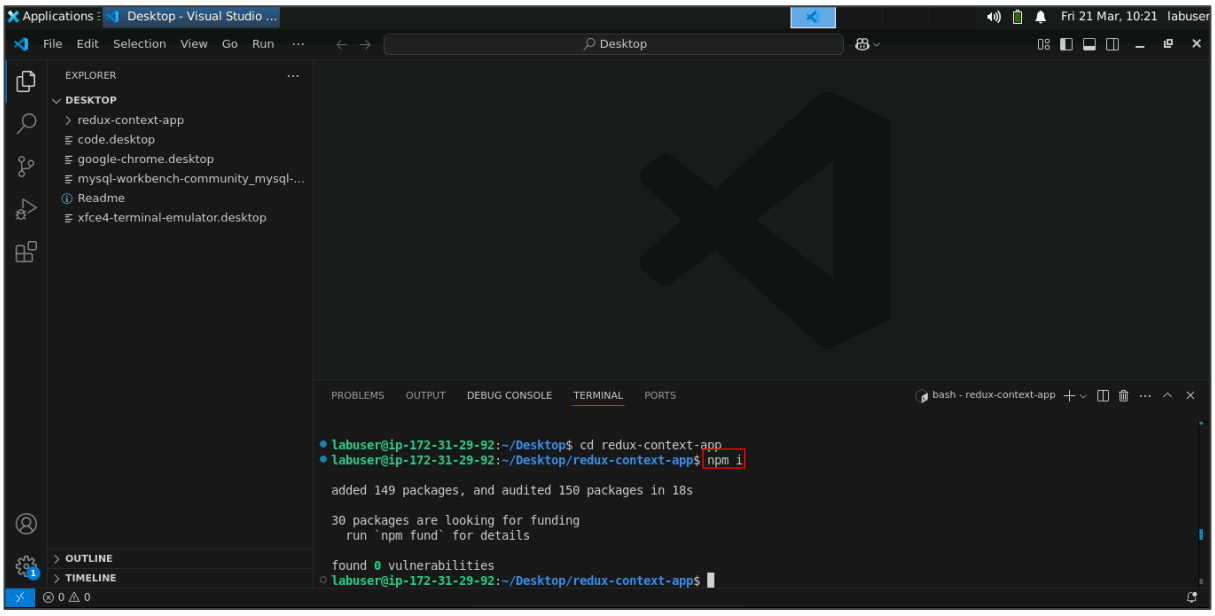
1.5 Run the following command to navigate to the project folder:

cd redux-context-app



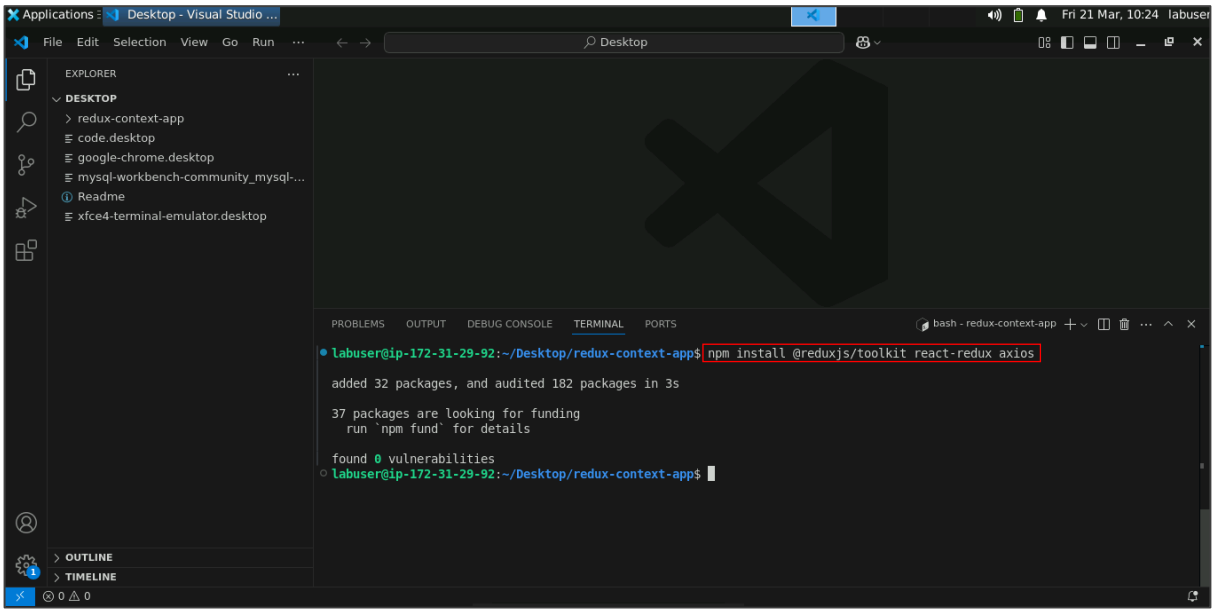
1.6 Run the following command to install dependencies:

npm i



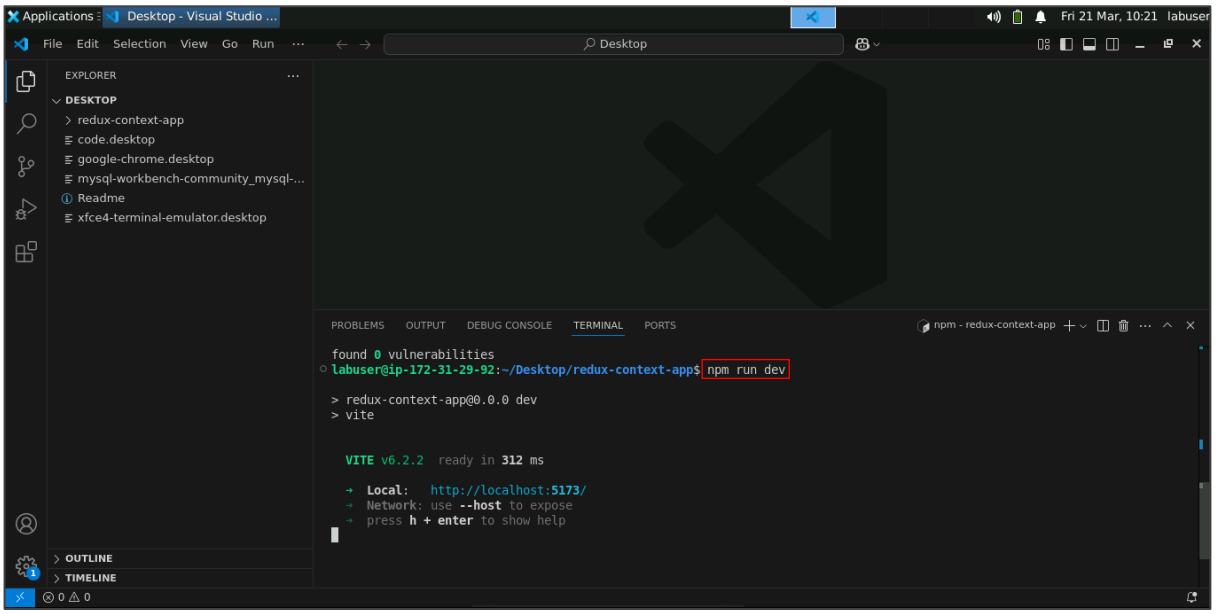
1.7 Run the following command to install @reduxjs/toolkit, react-redux, and axios:

npm install @reduxjs/toolkit react-redux axios



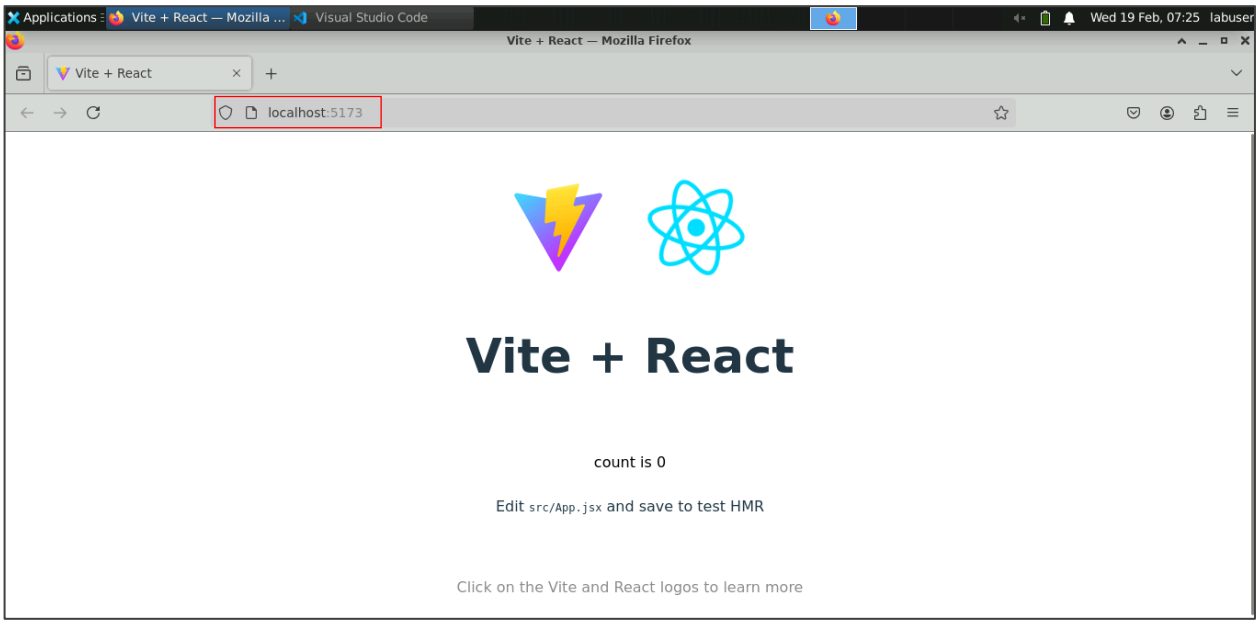
1.8 Run the following command to start the development server:

npm run dev



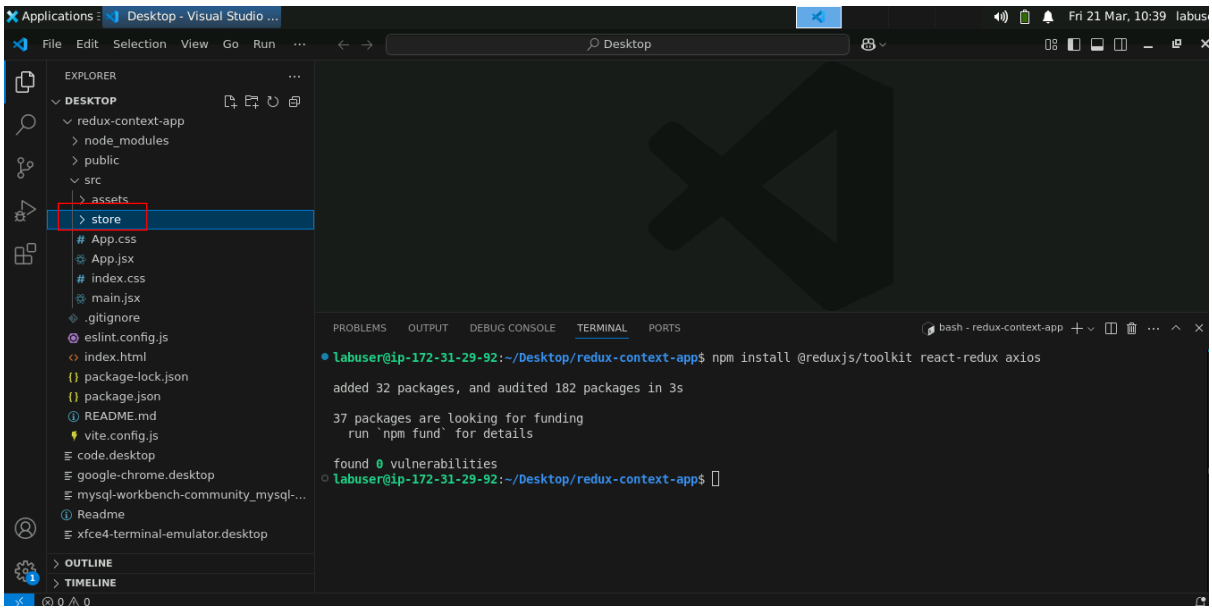
1.9 Navigate to the web browser and paste the link below:

http://localhost:5173

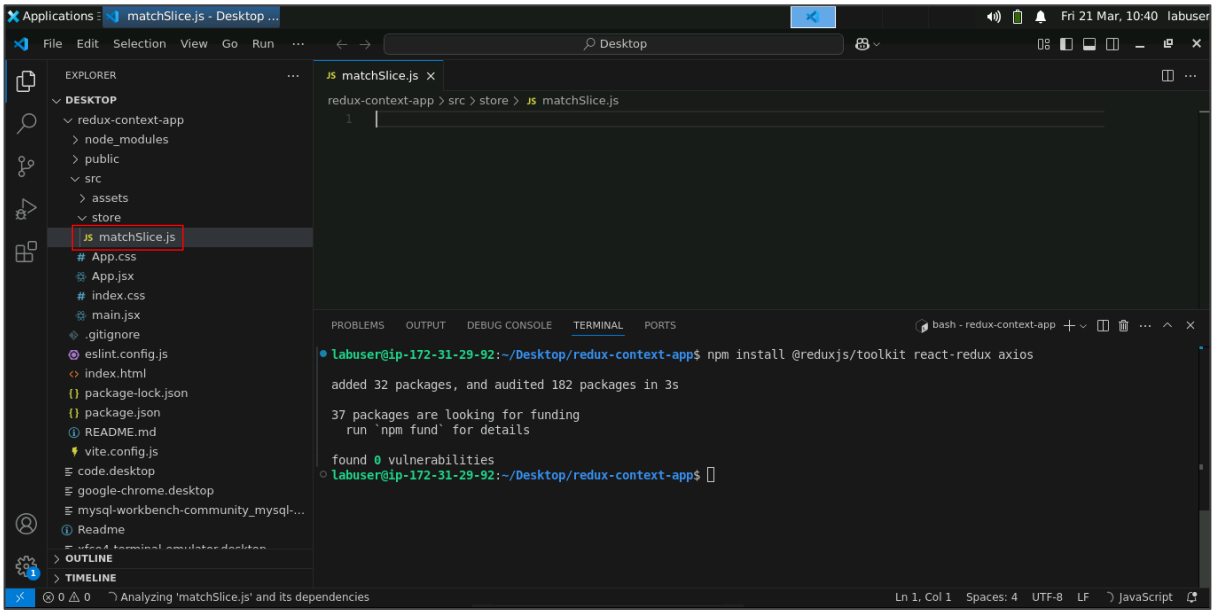


Step 2: Set up Redux store with slices and async thunks

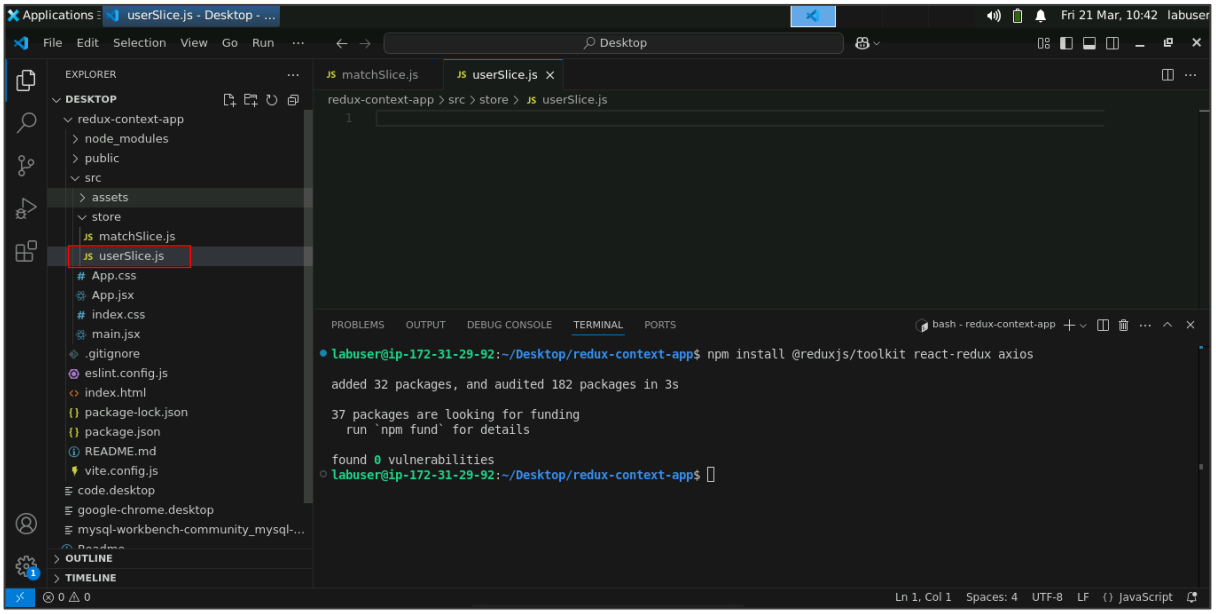
2.1 Inside the **src** folder, create a new folder named **store**



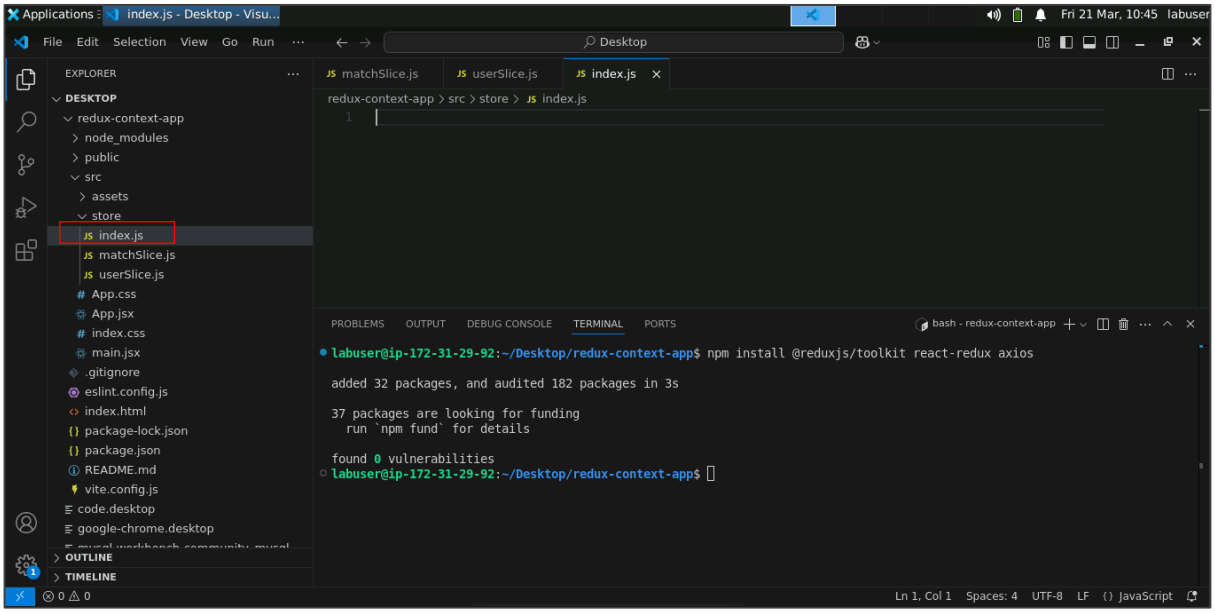
2.2 Inside the **store** folder, create a new file named **matchSlice.js**



2.3 Inside the **store** folder, create a new file named **userSlice.js**



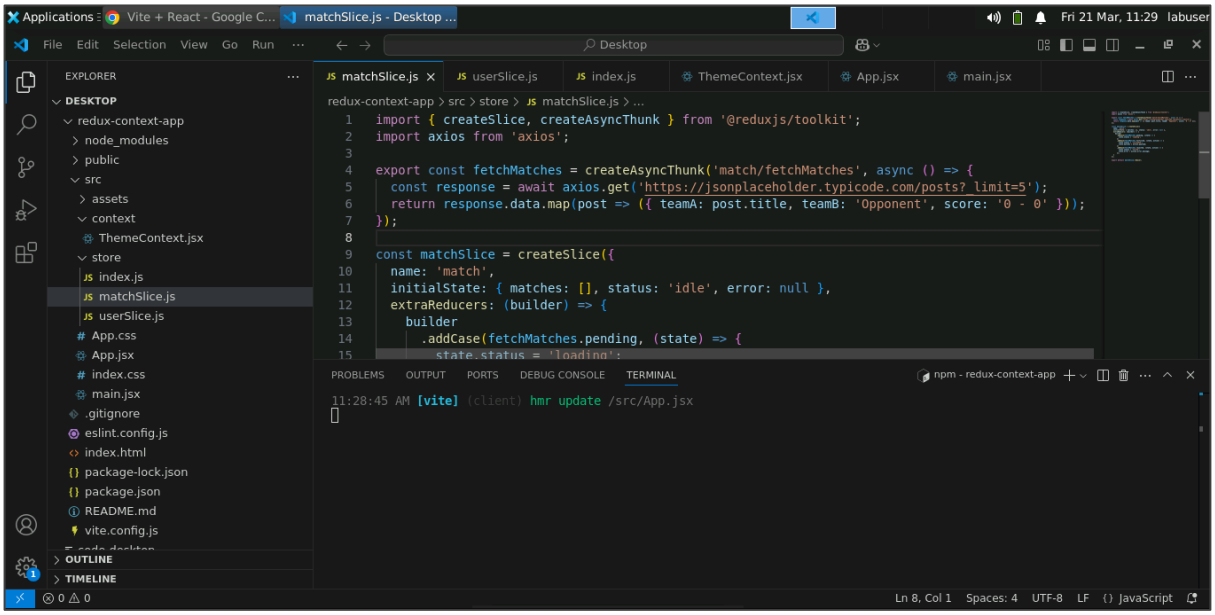
2.4 Inside the **store** folder, create a new file named **index.js**



2.5 Paste the code below in **matchSlice.js**:

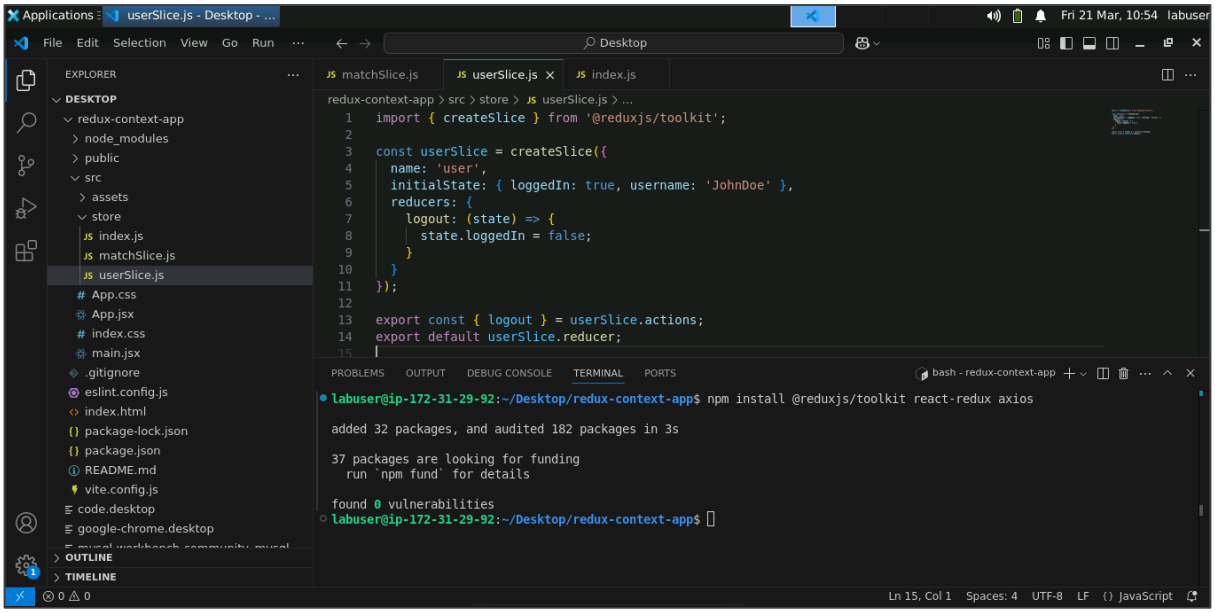
```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axios from 'axios';

export const fetchMatches = createAsyncThunk('match/fetchMatches', async ()
=> {
  const response = await
axios.get('https://jsonplaceholder.typicode.com/posts?_limit=5');
  return response.data.map(post => ({ teamA: post.title, teamB: 'Opponent',
score: '0 - 0' }));
});

const matchSlice = createSlice({
  name: 'match',
  initialState: { matches: [], status: 'idle', error: null },
  extraReducers: (builder) => {
    builder
      .addCase(fetchMatches.pending, (state) => {
        state.status = 'loading';
      })
      .addCase(fetchMatches.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.matches = action.payload;
```

```
    })
    .addCase(fetchMatches.rejected, (state, action) => {
      state.status = 'failed';
      state.error = action.error.message;
    });
  }
});

export default matchSlice.reducer;
```



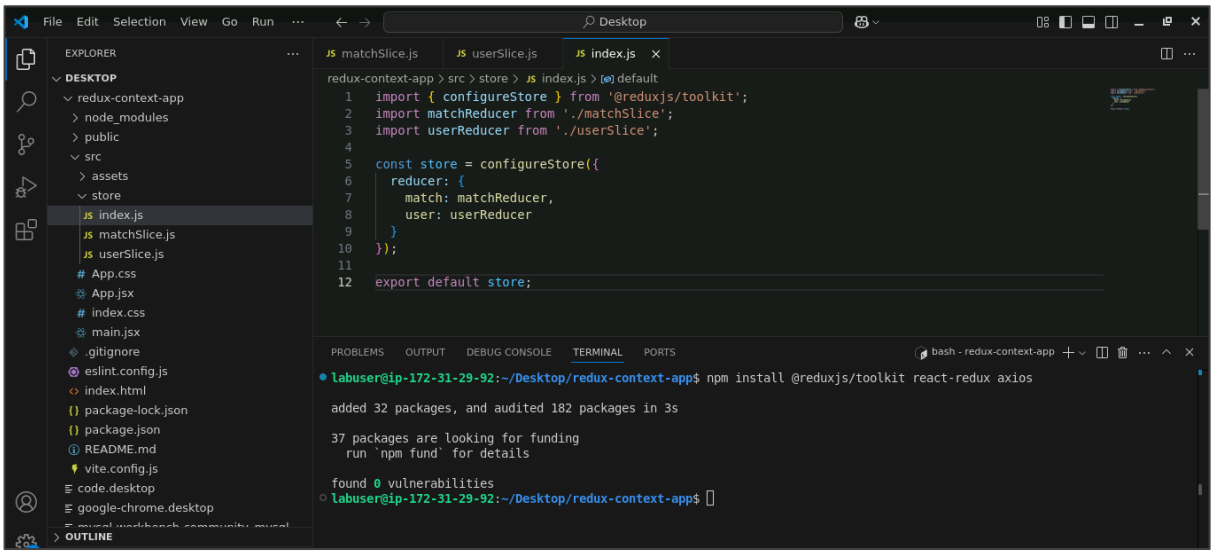
2.6 Paste the code below in **userSlice.js**:

```
import { createSlice } from '@reduxjs/toolkit';

const userSlice = createSlice({
  name: 'user',
  initialState: { loggedIn: true, username: 'JohnDoe' },
  reducers: {
    logout: (state) => {
      state.loggedIn = false;
    }
  }
});

export const { logout } = userSlice.actions;
export default userSlice.reducer
```



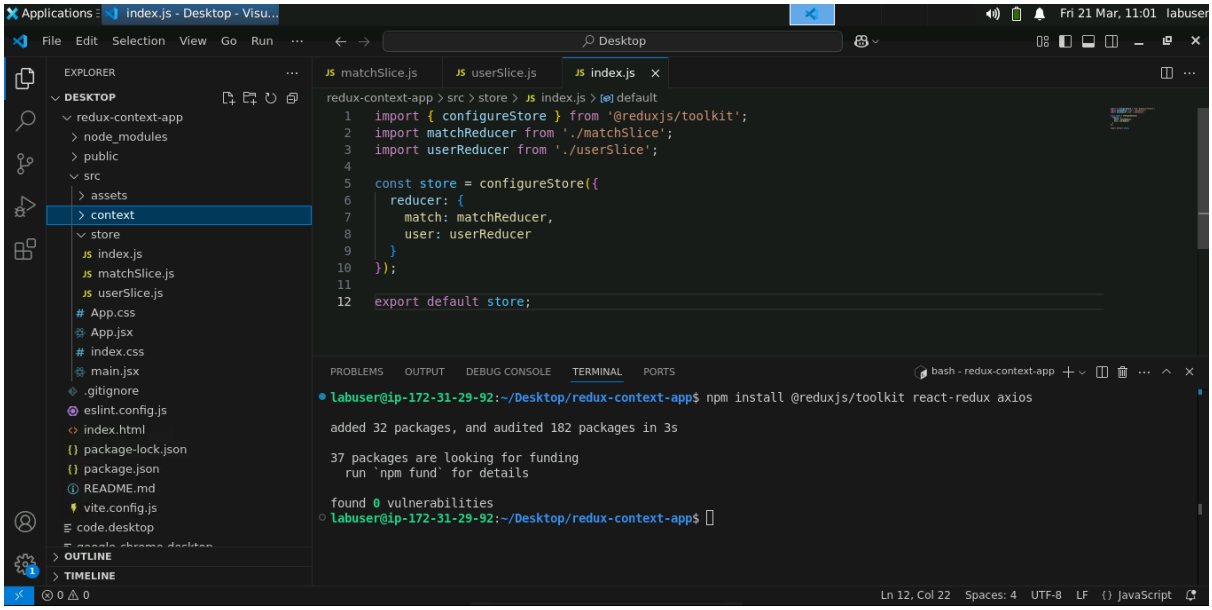
2.7 Paste the code below in **index.js**:

```
import { configureStore } from '@reduxjs/toolkit';
import matchReducer from './matchSlice';
import userReducer from './userSlice';

const store = configureStore({
  reducer: {
    match: matchReducer,
    user: userReducer
  }
});
export default store;
```

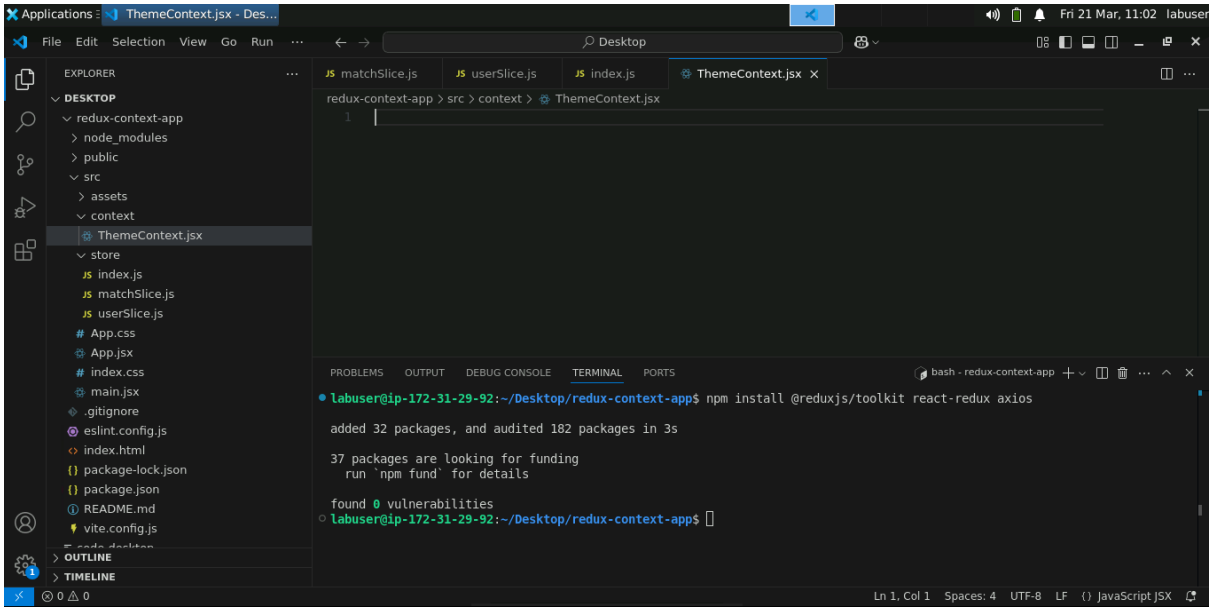


Step 3: Implement Context API for theme toggling

3.1 Inside the **src** folder, create a new folder named **context**



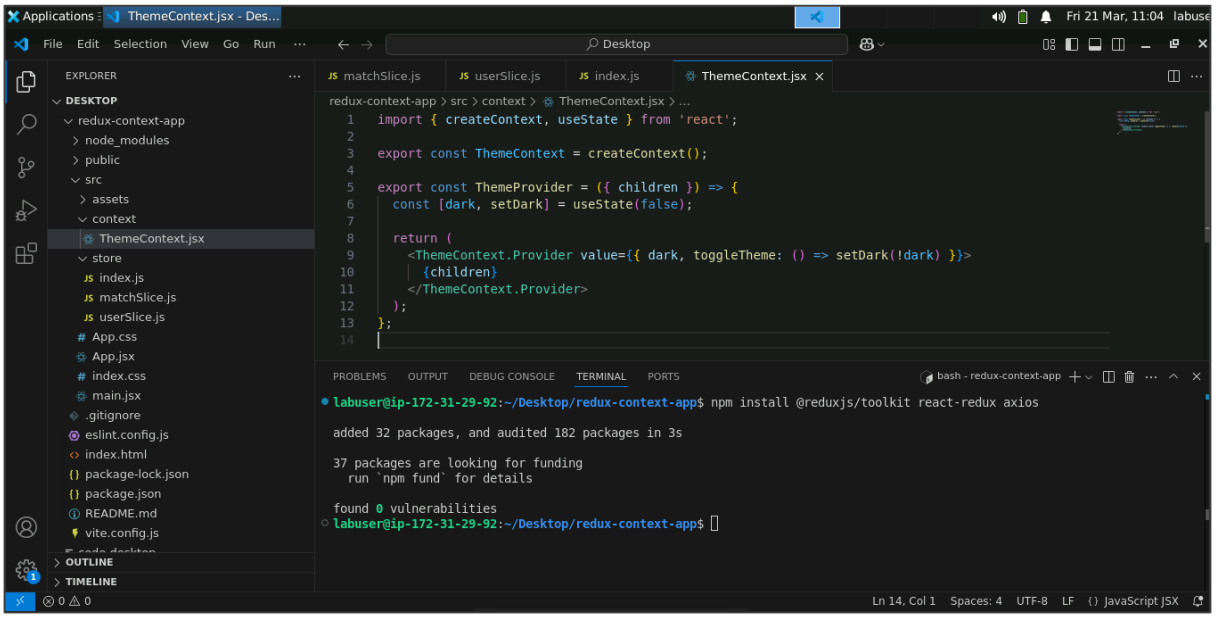
3.2 Inside the **context** folder, create a new file named **ThemeContext.jsx**



3.3 Paste the code below in **ThemeContext.jsx**:

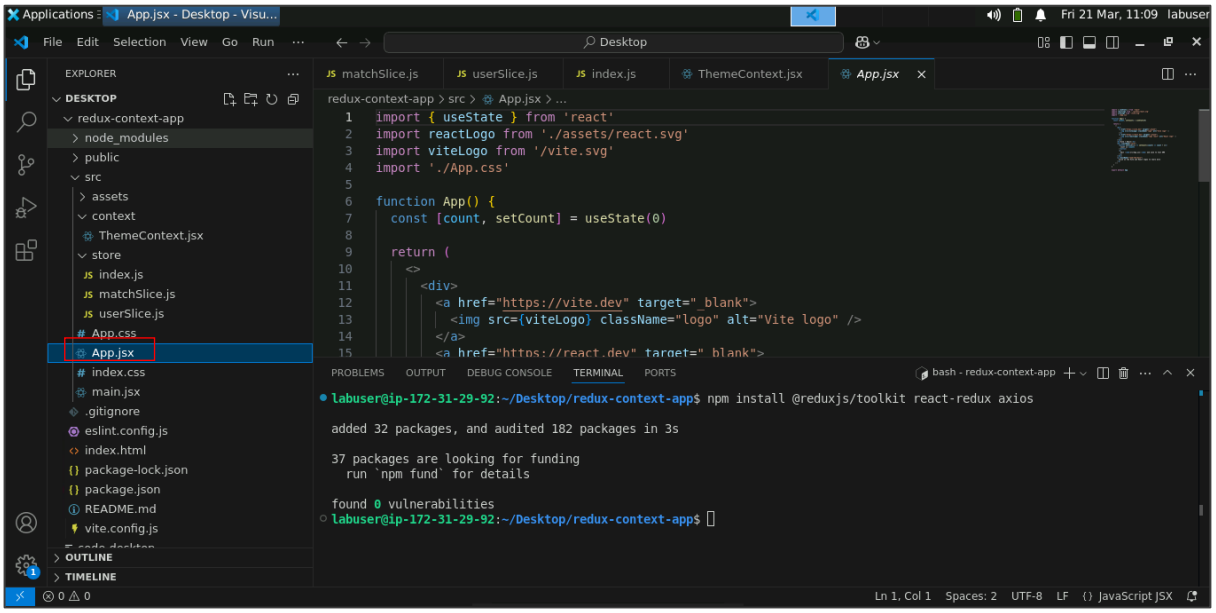
```
import { createContext, useState } from 'react';

export const ThemeContext = createContext();

export const ThemeProvider = ({ children }) => {
```

```
  const [dark, setDark] = useState(false);

  return (
    <ThemeContext.Provider value={{ dark, toggleTheme: () => setDark(!dark) }}>
      {children}
    </ThemeContext.Provider>
  );
};
```



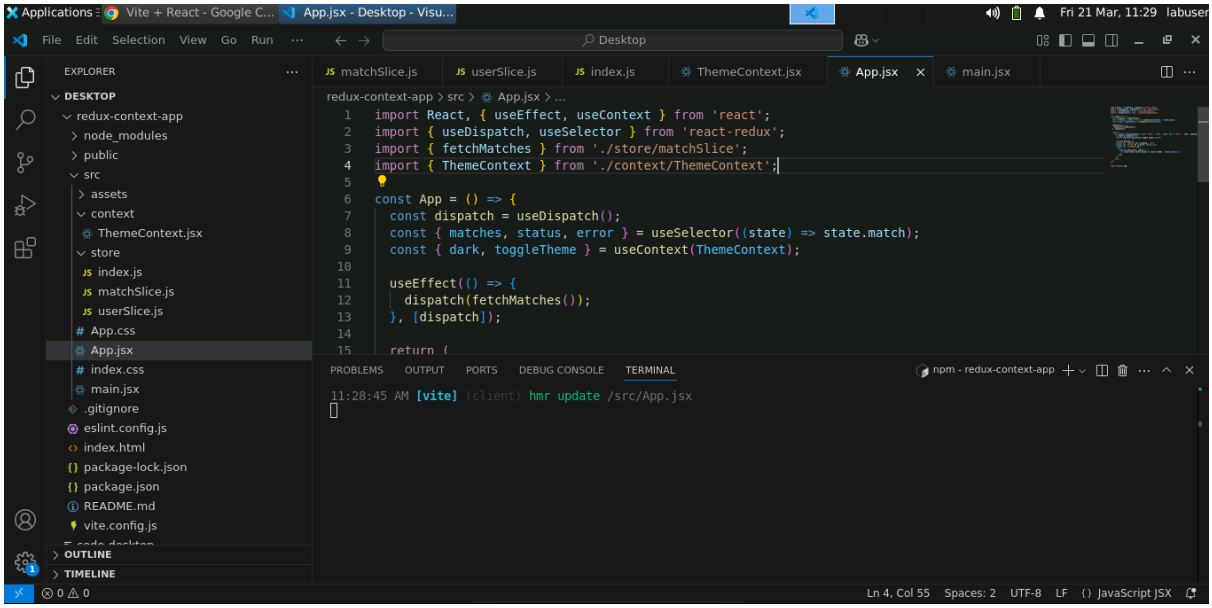
Step 4: Build App component with Redux and Context integration

4.1 Open the **App.jsx** file inside the **src** folder



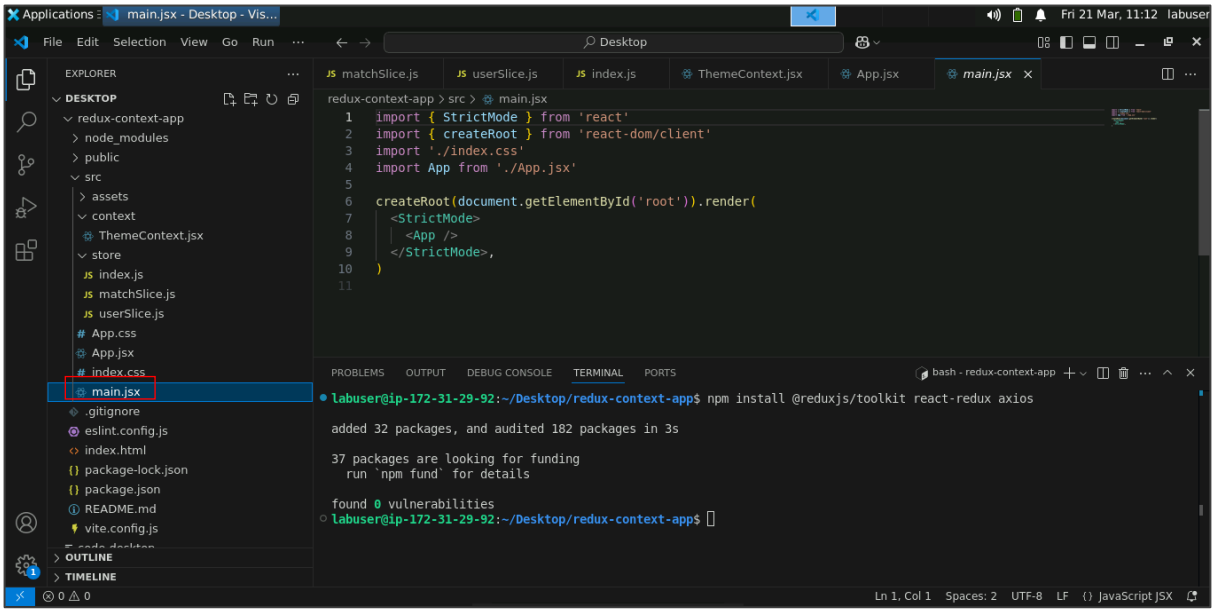
4.2 Paste the code below to include the new components in **App.jsx**:

```
import React, { useEffect, useContext } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchMatches } from './store/matchSlice';
import { ThemeContext } from './context/ThemeContext';

const App = () => {
  const dispatch = useDispatch();
  const { matches, status, error } = useSelector((state) => state.match);
  const { dark, toggleTheme } = useContext(ThemeContext);

  useEffect(() => {
    dispatch(fetchMatches());
  }, [dispatch]);

  return (
    <div style={{ backgroundColor: dark ? '#111' : '#fff', color: dark ? '#fff' : '#000',
padding: '20px' }}>
      <h1>Sports Dashboard</h1>
      <button onClick={toggleTheme}>Toggle Theme</button>

      <h2>Live Matches</h2>
      {status === 'loading' && <p>Loading...</p>}
      {status === 'failed' && <p>Error: {error}</p>}
      {status === 'succeeded' && (
        <ul>
          {matches.map((match, index) => (
            <li key={index}>{match.teamA} vs {match.teamB} - {match.score}</li>
          ))}
        </ul>
      )}
    </div>
  );
};

export default App;
```



Step 5: Update main.jsx

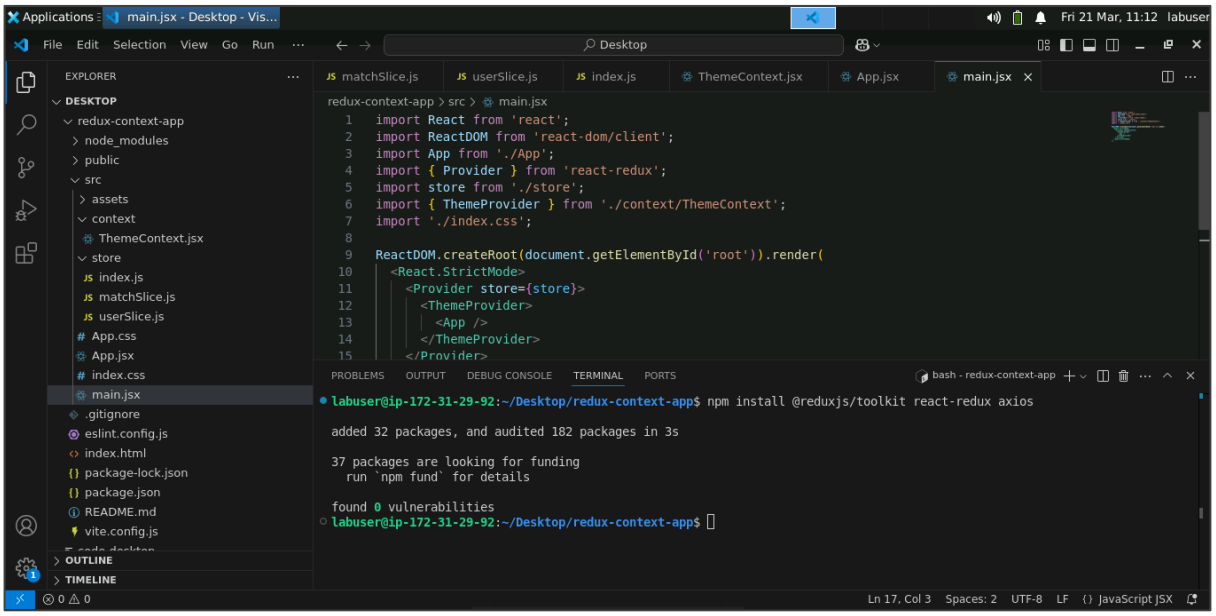
5.1 Open the **main.jsx** file inside the **src** folder



5.2 Paste the code below to include the new components in the **main.jsx**:

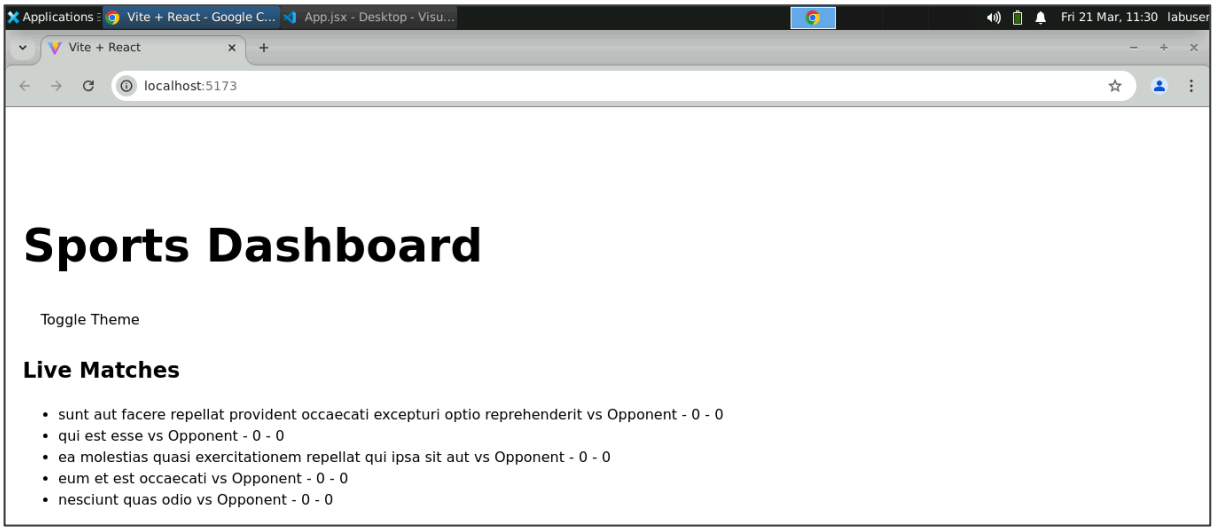
```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import { Provider } from 'react-redux';
import store from './store';
import { ThemeProvider } from './context/ThemeContext';
import './index.css';
```

```
ReactDOM.createRoot(document.getElementById('root')).render(
 <React.StrictMode>
  <Provider store={store}>
   <ThemeProvider>
    <App />
   </ThemeProvider>
  </Provider>
 </React.StrictMode>
);
```

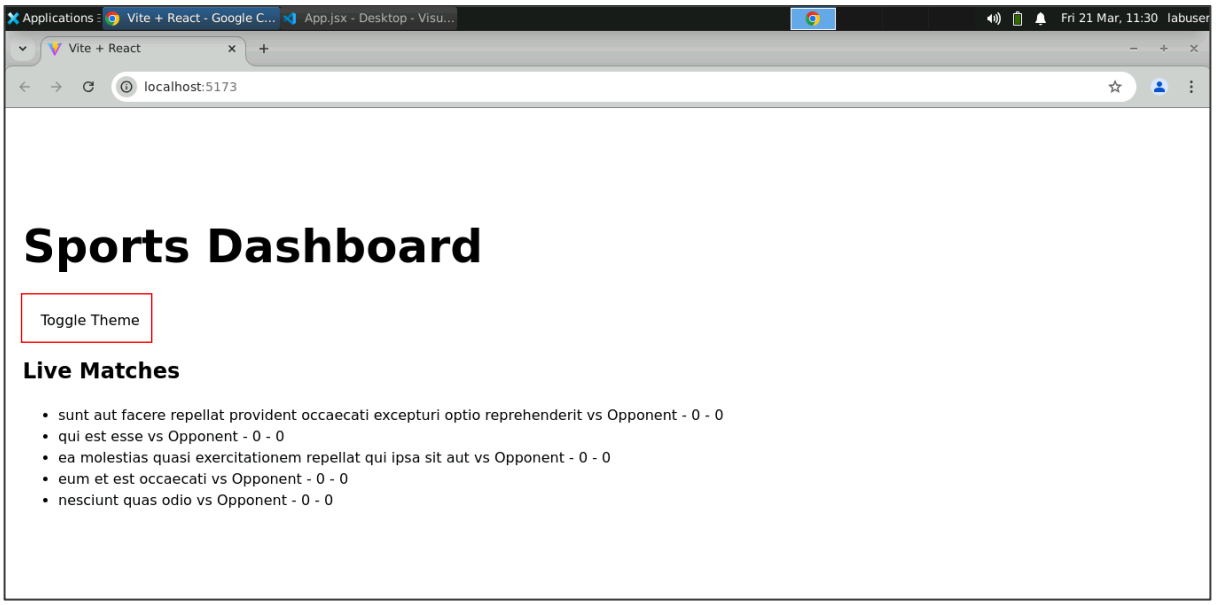


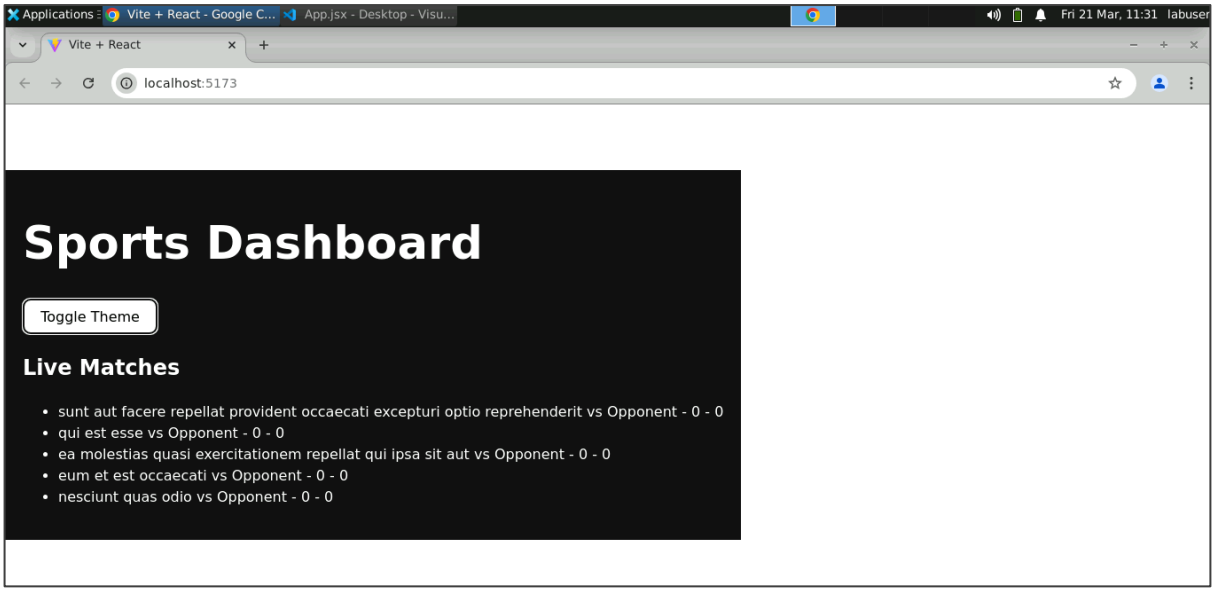
Step 6: Run and verify the application

6.1 Navigate to the browser and refresh the webpage



6.2 Click **Toggle Theme** to switch between light and dark themes





By following the above steps, a React application was created using Vite that integrates Redux Toolkit for state management, Context API for theme toggling, and createAsyncThunk for asynchronous data fetching. The project includes a sports dashboard displaying live match data, user session handling, and a toggle for light and dark themes.